Thought Points and Divergence as the Basis for Model Self-Continuous Learning

Zhenglong Qiu*

https://github.com/QZL2004qzl/DATP_DAM

Abstract

Current deep learning models face two fundamental bottlenecks. First, model parameters are fixed after pre-training, making it impossible to perform real-time, sustainable self-learning during deployment. When encountering new knowledge, conventional approaches such as fine-tuning and Retrieval-Augmented Generation (RAG) offer partial relief, but fine-tuning still risks catastrophic forgetting at considerable computational cost, while context embedding methods are constrained by context window length and produce no durable parameterized knowledge growth. Second, the Transformer-based self-attention mechanism incurs $O(n^2)$ time complexity and $O(n)$ memory consumption; as sequence length grows, inference cost expands quadratically, severely limiting the application of models in long-text and resource-constrained scenarios. To address this dual challenge, this paper draws on the Dual-Process Theory from human cognitive science and proposes the Divergent-Aggregate Thought-Point Network (DATP) and the Divergence Aggregation Mechanism (DAM). DATP achieves awareness of the model's cognitive boundaries through a decoupled judge network and introduces a dynamic thought-point pool with spatial aggregation and biological forgetting characteristics in non-parametric space, endowing the model with streaming, continual self-learning capability while keeping the base model parameters frozen. DAM replaces the conventional similarity metric with divergence, compressing historical temporal features into dynamically evolving thought points. It achieves

*E-mail: 2922490672@qq.com

$O(nd)$ time complexity and reduces inference memory from $O(nd)$ (which grows linearly with sequence length) under standard attention to a constant $O(d)$ level independent of sequence length n (requiring only two d-dimensional cumulative vectors that do not grow with generation steps), thus providing an efficient alternative to the attention mechanism. This paper theoretically demonstrates the scientific validity and feasibility of the thought point and divergence concepts, and constructs a complete mathematical framework along three dimensions: linear memory anchoring, implicit divergence gating, and evolutionary dynamics. It should be noted that the experimental validation in this paper is currently limited to small-scale regression and language modeling tasks; the deployment of DATP on large-scale language models and the comprehensive validation of DAM on more benchmarks are priorities for follow-up work. Pre-training experiments on the WikiText-103 dataset demonstrate that the DAM architecture achieves a perplexity of 20.69 at 84MB parameters and 1024 sequence length. DATP experiments on five standard regression datasets show that the number of thought points in full-evolution mode is reduced by approximately 72% (weighted by total count) compared to the no-evolution mode, with inference latency controlled at 0.08–0.63 milliseconds per sample, while regression performance remains stable. The work presented in this paper provides new theoretical foundations and engineering pathways for addressing the problem of model self-continuous learning.

Keywords: Thought Point; Divergence; Continual Learning; Attention Mechanism Replacement; Divergent-Aggregate Thought-Point Network; Divergence Aggregation Mechanism; Evolutionary Dynamics; Catastrophic Forgetting

Introduction

Core Dilemmas of Deep Learning Models

In recent years, large-scale deep learning models have achieved breakthrough progress in natural language processing, computer vision, and multimodal understanding. Large Language Models (LLMs), represented by the GPT series, Claude series, and DeepSeek series, have demonstrated remarkable emergent capabilities, reaching or even surpassing human expert levels in tasks such as machine translation, text generation, code writing, and complex reasoning. However, behind these brilliant achievements lie two fundamental architectural bottlenecks that have yet to be adequately addressed.

The first bottleneck is that models cannot achieve self-continuous learning after deployment. The current mainstream deep learning paradigm follows a

three-stage pipeline of "pre-training–fine-tuning–deployment." Once a model has been pre-trained and deployed to a production environment, its parameters are completely frozen. This means that: (1) any new knowledge, patterns, or concepts encountered by the model during inference cannot be absorbed and internalized, and can only be incorporated into the parameter space through the next expensive full or partial retraining cycle; (2) when facing Out-of-Distribution (OOD) samples, the model lacks adaptive, real-time error correction capability and can only continue outputting erroneous predictions; (3) the model's knowledge remains frozen at the cutoff date of the pre-training data, unable to achieve incremental knowledge accumulation and real-time evolution through continuous interaction with the environment, as biological agents do.

This "train–freeze–deploy" paradigm stands in stark contrast to how biological intelligence learns. The human brain continuously absorbs new information, corrects erroneous beliefs, and forgets ineffective knowledge in daily life, without ever needing to return to an offline phase for "whole-brain retraining." This continuous, streaming self-learning capability is a key ingredient on the path toward Artificial General Intelligence (AGI). However, achieving continual learning faces a fundamental technical challenge—Catastrophic Forgetting [20]: when a neural network undergoes gradient updates on new data, the weight configurations corresponding to old knowledge are irreversibly overwritten, causing the model's performance on old tasks to degrade sharply even as it improves on new ones.

The second bottleneck is the quadratic growth of computational and memory complexity in the Transformer self-attention mechanism. Since its introduction by Vaswani et al. in 2017 [1], the Transformer architecture has become the de facto standard in deep learning. Its core innovation—Scaled Dot-Product Self-Attention—achieves effective global context modeling by computing the similarity between every pair of tokens in the sequence. However, this fully-connected pairwise interaction mechanism incurs $O(n^2)$ time complexity and $O(n)$ Key-Value (KV) cache space complexity, where n is the input sequence length. This means that: (1) when sequence length grows from 1K to 32K, the computational load of a single attention layer increases by a factor of roughly 1024; (2) during autoregressive decoding, each newly generated token must attend to all historical tokens, and the size of the KV cache grows linearly with the number of generation steps; (3) in resource-constrained edge devices and mobile scenarios, this complexity growth renders long-text inference practically infeasible.

More fundamentally, these two bottlenecks are not isolated but tightly

intertwined. Under the standard Transformer architecture, if a model wishes to achieve continual learning, it must update the parameters of its attention layers, which not only triggers catastrophic forgetting but also makes the computational cost of each adaptive update unacceptable due to the quadratic complexity of the attention mechanism. Therefore, to achieve genuine model self-continuous learning, one must simultaneously address three levels of questions—"what to learn, how to learn, and where to learn"—and the answers to these three questions point respectively to: the form of knowledge representation (thought points), the basis for update decisions (divergence), and an efficient underlying computational operator (the DAM mechanism).

Inspiration from Cognitive Science: Dual-Process Theory

The methodological framework proposed in this paper draws its core inspiration from the Dual-Process Theory in cognitive science. This theory was systematically articulated by Nobel laureate in Economics Daniel Kahneman in his book *Thinking, Fast and Slow* (2011) [16], with its theoretical foundations laid by Stanovich and West in their 2000 work [17]. Dual-Process Theory divides human cognitive processes into two subsystems that are distinctly different in function, speed, and resource consumption:

- **System 1 (Fast Thinking)**: An automatic, intuitive, rapid-response system that requires almost no cognitive resources. It makes immediate judgments for most routine, familiar situations based on past experience and pattern matching. System 1 processing is parallel, unconscious, and can be completed at the millisecond level.
- **System 2 (Slow Thinking)**: A controlled, analytical, deep-reasoning system that demands substantial cognitive resources. It is activated only when System 1 encounters cognitive conflict, uncertainty, or highly complex problems, and engages in deliberate judgment through logical reasoning, causal analysis, and hypothesis testing. System 2 processing is serial, conscious, and requires subjective effort.

The mapping of this cognitive framework onto artificial intelligence has drawn increasing attention. Nye et al. (2021) [18] explicitly pointed out that current large language models are essentially "System-1-only" reasoners—they perform rapid pattern matching and statistical association on massive training data, but often exhibit systematic biases and hallucinations when facing tasks that require multi-step logical verification, constraint satisfaction, or counterfactual reasoning. In recent years, researchers have begun exploring pathways to embed Dual-Process Theory into AI architectures, including Neuro-Symbolic Reasoning systems, the SOFAI (Slow and Fast AI) multi-agent framework, and the use of Chain-of-Thought prompting to simulate

System 2 reasoning processes.

The work presented in this paper goes a step further: we not only treat Dual-Process Theory as a behavioral-level inspiration, but directly encode it into the foundational architecture design of neural networks. Within the DATP framework, the frozen-parameter universal base network serves as the intuitive System 1, while the dynamically evolving thought-point pool plays the role of System 2, activated when needed. The switch between the two is determined by a decoupled judge network (a cognitive meta-monitor) based on divergence measurement, thereby realizing a **Compute-on-Demand** cognitive resource allocation strategy.

Main Contributions

The main contributions of this paper can be summarized in the following four aspects:

1. **Conceptual innovation**: We provide the first mathematical definitions of "thought point" and "divergence," systematically translating concepts from human cognitive science—Dual-Process Theory, memory compression, and cognitive conflict detection—into fundamental computational primitives for deep learning, offering a new theoretical perspective for model self-continuous learning.
2. **Architectural innovation**: We design the Divergent-Aggregate Thought-Point Network (DATP), which, through its tripartite architecture of a universal base network, a decoupled judge, and a dynamic thought-point pool, achieves streaming continual learning without modifying base model parameters. We propose an evolutionary dynamics mechanism incorporating spatial aggregation and biological forgetting, ensuring the generalization capacity and boundedness of the thought-point pool.
3. **Operator innovation**: We propose the Divergence Aggregation Mechanism (DAM), which replaces similarity with divergence as the core metric for information interaction, reducing the $O(n^2)$ complexity of self-attention to $O(nd)$, and reducing inference memory from $O(nd)$ (which grows linearly with sequence length) under standard attention to $O(d)$ independent of sequence length (requiring only two d-dimensional cumulative vectors). We theoretically prove the information retention capacity and computational efficiency advantages of this mechanism in long-text scenarios.
4. **Empirical validation**: We conduct systematic experimental validation on the WikiText-103 language modeling benchmark and five standard regression datasets,

demonstrating that DAM achieves linear complexity while maintaining language modeling capability comparable to the standard attention mechanism, and that DATP compresses the number of thought points by roughly 75% through evolutionary dynamics while maintaining stable predictive performance.

Paper Organization

The remainder of this paper is organized as follows: Section 2 systematically reviews related work, covering KV cache optimization, attention mechanism alternatives, Generative Adversarial Networks, and Mixture of Experts models; Section 3 presents the proposed methods in detail, including the complete mathematical framework, PyTorch reference implementations, and architectural design of the Divergent-Aggregate Thought-Point Network (DATP) and the Divergence Aggregation Mechanism (DAM); Section 4 reports experimental settings and result analysis; Section 5 provides an in-depth discussion of the experimental results and methodological limitations; Section 6 concludes the paper and proposes future research directions.

Related Work

KV Cache Optimization: MQA, GQA, and MLA

During the inference of autoregressive language models, the KV cache (Key-Value Cache) is the primary source of memory consumption. For a Transformer model with L layers, h attention heads, and hidden dimension d, processing a sequence of length n, the total size of the KV cache is:

$$\text{KV-Cache} = 2 \times L \times h \times d_h \times n \times \text{precision} \tag{1}$$

where $d_h = d/h$ is the per-head dimension. Taking LLaMA2-7B as an example ($L = 32$, $h = 32$, $d_h = 128$), under FP16 precision each generated token requires approximately 0.5 MB of KV cache; when processing long sequences in batches, the KV cache quickly exhausts the GPU's High Bandwidth Memory (HBM).

Multi-Query Attention (MQA) [2], proposed by Shazeer in 2019, was among the earliest systematic solutions targeting the KV cache problem. The core idea of MQA is to have all attention heads share a single set of Key and Value projection matrices while maintaining independent Query projections. This design reduces the KV cache size from $O(h \times n)$ directly to $O(1 \times n)$, i.e., a compression ratio of $1/h$. MQA has been

widely adopted in models such as PaLM, StarCoder, and Gemini. However, the cost of MQA is a certain degradation in the expressive power of attention—all heads are forced to use the same key-value representation, which may cause performance degradation on complex tasks requiring diverse attention patterns.

Grouped-Query Attention (GQA) [3], proposed by Ainslie et al. in 2023, finds a compromise between MHA and MQA. GQA partitions the h query heads into g groups ($1 < g < h$), with each group sharing one set of Key-Value projections. When $g = h$, it degenerates to MHA; when $g = 1$, it degenerates to MQA. LLaMA2-70B adopts a GQA configuration with $g = 8$, achieving roughly 8× KV cache compression while maintaining near-identical performance to MHA on long-text tasks. LLaMA3, ChatGLM2/3, and DeepSeek-V1 have also widely adopted GQA.

Multi-Head Latent Attention (MLA) [4], proposed by the DeepSeek team in DeepSeek-V2 (2024), represents the latest paradigm shift in KV cache optimization. MLA does not reduce the KV cache through head sharing, but instead jointly compresses Key and Value into a low-dimensional latent space. Specifically, MLA maps the input hidden state to a latent vector $\mathbf{c}$ of dimension d_c through a down-projection matrix W^{DKV}, caches only this latent vector during inference, and reconstructs Key and Value on demand through up-projection. Through the matrix absorption technique (absorbing the up-projection into the Query projection), MLA eliminates the computational overhead of explicit decompression during inference. DeepSeek-V2 uses a latent dimension of $d_c = 512$, achieving approximately 30× KV cache compression, thereby supporting an ultra-long context window of 128K tokens.

Table 1 Comparison of KV cache optimization approaches

Method	KV Cache Copies	Compression Ratio
MHA (Vaswani et al., 2017) [1]	h	1×
MQA (Shazeer, 2019) [2]	1	$1/h$
GQA (Ainslie et al., 2023) [3]	g	g/h
MLA (DeepSeek, 2024) [4]	$d_c + d_{\text{rope}}$	∼10–30×

Alternatives to the Attention Mechanism: From Linear Attention to State Space Models

Substantial exploration has been conducted in academia on reducing the computational complexity of attention. These efforts can be broadly categorized into linear

attention methods, State Space Models (SSMs), and hybrid architectures.

Linear attention methods are built on the core idea of exploiting kernel function properties to reorder the attention computation from the "dot-product-then-weight" sequence of $(QK^T)V$ to the "weight-then-dot-product" sequence of $Q(K^TV)$, thereby reducing complexity from $O(n^2)$ to $O(nd^2)$ or $O(nd)$. The Linear Transformer was first proposed by Katharopoulos et al. in 2020 [5], using the ELU activation function plus 1 as the kernel function to ensure the non-negativity of Key projections. Performer (Choromanski et al., 2021) [6] employs orthogonal random features to approximate the softmax kernel, achieving competitive performance with standard Transformers on multiple benchmarks. Gated Linear Attention (GLA, Yang et al., ICML 2024) [24] introduces a data-dependent gating mechanism and a hardware-efficient FlashLinearAttention implementation, demonstrating competitive capability with the LLaMA architecture on language modeling tasks. Hedgehog (Zhang et al., 2024) [26] achieves high-quality approximation of standard attention by linear attention through a trainable softmax mimicry mechanism. BASED (Arora et al., 2024) [25] combines linear attention with sliding-window attention, maintaining recall performance comparable to FlashAttention-2 while achieving a 24× throughput improvement.

State Space Models (SSMs) constitute another important technical direction. The theoretical foundation of SSMs can be traced back to linear time-invariant systems in control theory, with the core idea of using a latent state vector to dynamically encode the history of the sequence. The S4 model proposed by Gu et al. [7] first applied structured state spaces to long-sequence modeling. Mamba (Gu & Dao, 2023) [8] broke the linear time-invariant limitation of S4 by introducing an input-dependent selective mechanism (Selective SSM), making the state transition matrix A, input projection matrix B, and output projection matrix C all functions of the input, substantially enhancing the contextual modeling capacity of SSMs. Mamba-2 (Dao & Gu, 2024) [9] further revealed the deep mathematical connection between SSMs and linear attention—Structured State Space Duality (SSD)—and achieved 2–8× training acceleration through an optimized parallel scan algorithm. Jamba (Lieber et al., 2024) [31] proposed a hybrid Transformer-Mamba architecture that combines the expressive power of full attention layers with the long-sequence efficiency of Mamba layers. Retentive Network (RetNet, Sun et al., 2023) [30] introduced an exponentially decaying retention mechanism, achieving a competitive balance between training parallelism and inference efficiency. The RWKV series (Peng et al., 2023) [10] adopts an RNN-style linear recurrence mechanism, demonstrating performance competitive with Transformers on

language modeling tasks.

TTT (Test-Time Training) [11], proposed by Sun et al. in 2024, pioneers a fundamentally new sequence modeling paradigm. TTT layers treat the hidden state itself as a machine learning model (a linear model or MLP), and the state update rule is one step of gradient descent of that model on a self-supervised reconstruction loss. Since the hidden state continues to learn during inference (i.e., test-time training), TTT layers exhibit expressive capacity surpassing that of fixed-parameter RNNs while maintaining linear complexity. TTT-Linear demonstrates better perplexity than Mamba at the 125M–1.3B parameter scale, especially on sequences exceeding 32K in length.

Table 2 Complexity comparison of attention alternatives

Method	**Training Complexity**	**Inference Complexity**	**In**
Standard Attention (Transformer) [1]	$O(n^2d)$	$O(n^2d)$	
Linear Attention (Linear Transformer) [5]	$O(nd^2)$	$O(nd^2)$	
Mamba (S6) [8]	$O(nd)$	$O(nd)$	
Mamba-2 (SSD) [9]	$O(nd)$	$O(nd)$	
RWKV [10]	$O(nd)$	$O(nd)$	
TTT-Linear [11]	$O(nd)$	$O(nd)$	
DAM (This paper)	$O(nd)$	$O(nd)$	

Comparative analysis of DAM and other alternatives. Among the attention alternatives listed in Table 2, Linear Attention (Linear Transformer) needs to maintain a KV state matrix of size $O(d^2)$ during inference, representing moderate memory overhead; State Space Models and RNN-style models such as Mamba, Mamba-2, and RWKV compress the inference state to a fixed-dimensional vector of $O(d)$, placing them in the same order of magnitude as DAM. However, DAM possesses a unique advantage over these approaches: **DAM completely dispenses with matrix multiplication**. Standard attention relies on the QK^T matrix multiplication (computational complexity $O(n^2d)$); although linear attention reorders the computation, it still requires matrix multiplication in the form $Q(K^TV)$ ($O(nd^2)$); Mamba and RWKV likewise involve input-dependent linear projection matrix operations during state updates. In contrast, the core computation of DAM consists solely of element-wise multiplication ($\phi(K) \cdot V$), accumulation (cumsum), element-wise division ($\frac{\text{kv\_cumsum}}{\text{den\_cumsum}}$), and the Hadamard product ($\phi(Q) \odot \mathcal{M}_t$), requiring no matrix multiplication between Q and K whatsoever, resulting in a more streamlined computation path. In terms of inference memory,

DAM needs only to maintain two cumulative vectors of shape $[1, d]$ (kv_cumsum and den_cumsum), completely decoupled from sequence length n, with constant storage overhead of $O(d)$. Although RWKV and Mamba-2 also compress inference state to the $O(d)$ level, their state updates involve multiple matrix-vector multiplications; the power consumption and latency overhead of matrix multiplication engines cannot be ignored in batch inference and large-scale deployment scenarios. DAM achieves the same order of inference memory using purely element-wise operations, offering more pronounced efficiency advantages in resource-constrained edge devices and long-sequence inference scenarios.

GAN, MoE, and Reinforcement Learning: Borrowed Core Insights

The DATP methodological framework proposed in this paper conceptually integrates core insights from three domains—Generative Adversarial Networks (GANs), Mixture of Experts (MoE), and Reinforcement Learning (RL)—borrowing the error-correction intuition of GANs, the sparse activation advantages of MoE, and the interactive philosophy of RL, while mathematically realizing robust, positively-directed evolution during streaming inference through "residual in-situ embedding" and "ecological aggregation-forgetting" mechanisms.

Generative Adversarial Networks (GANs) [12], proposed by Goodfellow et al. in 2014, achieve adversarial training through a zero-sum game between a generator and a discriminator. Their core insight is that the discriminator serves as an external quality judge, driving the improvement of the generator's capability through feedback signals. This "error-correction intuition" is directly borrowed in the design of DATP—the judge network $g_\phi(\cdot)$ predicts the probability that the current base model will make a significant error, providing the decision basis for the embedding and activation of thought points. Unlike GANs, the judge in DATP does not engage in a zero-sum adversarial game with the base model, but rather functions as a **decoupled cognitive meta-monitor** providing auxiliary cognitive boundary awareness.

Mixture of Experts (MoE) [27, 13, 14] is built on the core idea of dynamically activating a small number of experts for each input through a gating network (Top-k sparse activation). MoE's inspiration for this paper is reflected at two levels: first, **sparse activation advantages**—the thought-point pool activates only the single most relevant thought point (or zero) for each inference step; second, **modular knowledge storage**—each thought point is an independent local expert, carrying error-correction

capability for a specific knowledge region. The fundamental difference between MoE and DATP is that in MoE, the number and parameters of experts are fixed during training and optimized through large-scale gradient descent, whereas DATP's thought points are **non-parametric**, **dynamically created and forgotten on demand** during inference, with learning accomplished through a few steps of online gradient descent.

Reinforcement Learning (RL) [15] is centered on the idea that an agent learns an optimal policy through interaction with the environment based on reward signals. RL's inspiration for this paper lies in the notion that an intelligent agent should judge the correctness of its own behavior through interaction with the external world and make adjustments accordingly. DATP concretizes this idea as a "model–judge–environment" triadic interaction loop, in which the "reward signal" is the residual vector between the true label y and the base prediction $\hat{y}_{\text{base}}$, enabling thought points to learn rich directional correction information.

Continual Learning and Catastrophic Forgetting

Continual Learning, also known as Lifelong Learning or Incremental Learning, aims to endow neural networks with the ability to continuously absorb new knowledge without forgetting previously acquired knowledge. The core challenge of this field is **Catastrophic Forgetting** [20]—when a network undergoes parameter updates on new data, the weight configurations corresponding to old tasks are overwritten. Wang et al. (2024) in their IEEE TPAMI paper "A Comprehensive Survey of Continual Learning" [19] categorize existing methods into three major families:

- **Regularization-based methods**: Such as Elastic Weight Consolidation (EWC, Kirkpatrick et al., 2017) [20] and Synaptic Intelligence (SI, Zenke et al., 2017) [22], which mitigate forgetting by imposing additional regularization penalties on important parameters.
- **Rehearsal-based methods**: Such as Experience Replay and Generative Replay, which maintain old knowledge while training on new tasks by storing or generating representative samples of old tasks.
- **Parameter Isolation-based methods**: Such as Progressive Networks (Rusu et al., 2016) [21] and PackNet (Mallya et al., 2018) [28], which completely avoid parameter overwriting by allocating independent parameter subspaces for new tasks.

Zheng et al. (2024) in "Towards Lifelong Learning of Large Language Models: A Survey" [23] further categorize LLM continual learning strategies into internal knowl-

edge updating (within parameter space) and external knowledge augmentation (outside parameter space), noting that external methods (such as RAG) avoid forgetting but depend on the quality and retrieval precision of external knowledge bases.

The DATP method presented in this paper belongs to the category of **non-parametric extensions of parameter isolation methods**: we do not modify base model parameters, but instead construct a new, independently evolving thought-point pool outside the parameter space. Unlike traditional parameter isolation methods, DATP's thought points are not pre-allocated for different tasks, but rather **automatically grow, aggregate, and prune** during streaming interaction, making them naturally suited to open-domain continual learning scenarios in non-stationary environments.

Proposed Method

This section constitutes the core of the paper, systematically presenting the complete methodological framework of the Divergent-Aggregate Thought-Point Network (DATP) and the Divergence Aggregation Mechanism (DAM). Section 3.1 begins by providing rigorous mathematical characterizations of thought points and divergence at the definitional level; Section 3.2 details the architectural design, algorithmic flow, and evolutionary dynamics of DATP; Section 3.3 delves into the mathematical principles, computational properties, and complexity analysis of the DAM mechanism.

Mathematical Definitions of Thought Points and Divergence

Formal Definition of Thought Points

Definition 3.1 (Thought Point)**.** Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the input feature space and $\mathcal{Y} \subseteq \mathbb{R}^k$ the output space. A thought point is a localized knowledge unit in feature space, formally defined as a triple:

$$tp_k = \langle \mathbf{a}_k, \mathbf{W}_k, H_k \rangle \tag{2}$$

where:

- $\mathbf{a}_k \in \mathbb{R}^d$ is the **spatial anchor feature**, which records the latent-space feature representation of the input sample at the time the thought point was created, serving as a reference for evaluating the divergence between new inputs and the thought point.
- $\mathbf{W}_k$ is the **local residual correction weight**, which stores the mapping capability

from input features to output residuals. Its parameter scale and structure can be flexibly designed according to task requirements (e.g., a single-layer linear transformation or a small MLP).

- $H_k \in [0, 1]$ is the **biological health score**, a scalar state value modeling biological synaptic plasticity and pruning that drives the lifecycle management of the thought point.

Definition 3.2 (Thought-Point Pool)**.** The thought-point pool $\mathcal{M}$ is the set of all currently active thought points:

$$\mathcal{M} = \{tp_1, tp_2, \ldots, tp_K\}, \quad K = |\mathcal{M}| \leq K_{\max} \tag{3}$$

where $K_{\max}$ is a pre-specified maximum capacity limit to prevent unbounded memory growth.

Why "thought point"? The naming is directly inspired by observations of human cognitive mechanisms. The Hippocampal Memory Indexing Theory (Teyler & DiScenna, 1986) [32] posits that the hippocampus does not directly store detailed perceptual memories, but rather stores index pointers to cortical regions, analogous to a distributed content-addressable memory. Sparse Coding Theory (Olshausen & Field, 1996) [33] reveals that neurons in the primary visual cortex exhibit sparse and localized responses to natural scenes. The "concept neurons" discovered by Quiroga et al. (2005) [34] (such as the famous "Jennifer Aniston neuron") provide direct neurophysiological evidence for discretized, compressed knowledge representations. Predictive Coding Theory (Rao & Ballard, 1999; Friston, 2005) [35, 36] describes the brain as a hierarchical prediction machine that updates its internal model when prediction diverges from actual input. Synthesizing these findings, the human knowledge representation system possesses three core properties: (1) **compressibility**—information is abstracted into compact neural representations; (2) **discreteness**—knowledge is organized into independently activatable units; (3) **divergence-driven**—knowledge updates occur only when there is significant divergence between prediction and input. We use the geometric metaphor of a "point" because, in high-dimensional latent space, each thought anchor represents a concrete location on the feature manifold, and the divergence (Euclidean distance) between thought points characterizes the geometric boundaries of knowledge regions.

Formal Definition of Divergence

Definition 3.3 (Divergence)**.** Given the latent-space feature $h_x \in \mathbb{R}^d$ of an input sample after the base feature extractor and the anchor feature $\mathbf{a}_k$ of thought point tp_k,

the **spatial divergence** between them is defined as:

$$D(h_x, tp_k) = \|h_x - \mathbf{a}_k\|_2 = \sqrt{\sum_{i=1}^{d}(h_{x,i} - a_{k,i})^2} \tag{4}$$

More generally, for sequence processing scenarios under the DAM mechanism, the **implicit divergence** is defined as the element-wise modulation between the activated Query and the condensed thought point:

$$\Phi(Q_t) = \mathrm{ELU}(Q_t) + 1.0 \tag{5}$$

Why "divergence"? In the conventional Transformer self-attention mechanism, the dot-product similarity between Query and Key is the core computational primitive, measuring "how similar the new input is to historical inputs." However, similarity metrics suffer from two fundamental problems: (1) **lack of directionality**—dot-product similarity can only indicate "how related," but cannot distinguish among three essentially different relationships: "positive correlation," "negative correlation," and "conflict"; (2) **scale sensitivity**—the magnitude of similarity is severely affected by feature norms, making it difficult to set a uniform threshold across different layers and heads. Divergence, by contrast, more precisely captures the **degree of inconsistency** or **cognitive conflict intensity** between the current input and existing knowledge. At the cognitive level, "divergence" inherently carries an action intention of "needing to be resolved"—high divergence triggers cognitive correction, while low divergence maintains the status quo. This conceptual shift from "similarity" to "divergence" is the key transition that upgrades attention from an "information retrieval" paradigm to a "cognitive error correction" paradigm.

The Cognitive Loop of Thought Points and Divergence

Thought points and divergence together form a complete cognitive closed-loop system. Figure 1 illustrates the logical flow of this cycle:

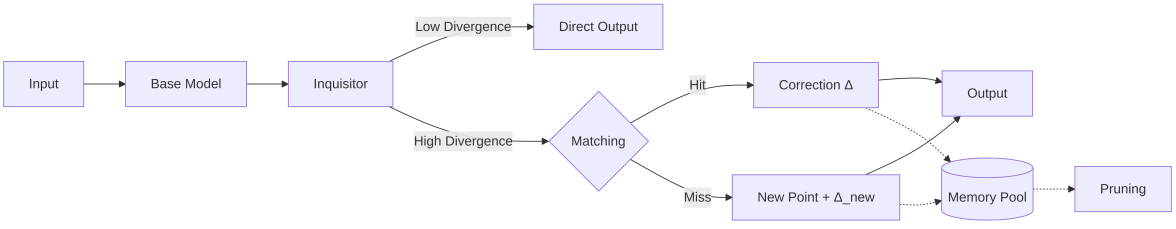


Figure 1 The cognitive closed loop of thought points and divergence

What makes this loop distinctive is that while traditional neural networks follow a unidirectional "learning → prediction" process, DATP realizes a cyclical process of

"prediction → evaluation → learning → re-prediction," in which each cycle produces a small expansion or correction of the model's knowledge boundary, thereby achieving positive knowledge accumulation through streaming interaction.

Divergent-Aggregate Thought-Point Network (DATP)

The Divergent-Aggregate Thought-Point Network (DATP) is a novel neural network architecture that endows a frozen base model with the capability for streaming, continual self-learning during the inference deployment phase. This section systematically presents the DATP methodology at five levels: overall architecture, base network design, judge mechanism, thought-point pool operations, and evolutionary dynamics.

Overall Architecture

DATP consists of three functionally decoupled core components:

1. **Universal base network f_θ (System 1)**: Responsible for extracting input features and producing baseline predictions. During deployment, its parameters θ are completely frozen.
2. **Decoupled judge network g_ϕ (Environment Proxy)**: Responsible for evaluating the quality of the base network's output, predicting the probability that the current input will cause the base model to make a significant error.
3. **Dynamic thought-point pool $\mathcal{M}$ (System 2)**: Composed of a set of dynamically created, online-learned, and adaptively evolving thought points, providing local residual correction when the base network makes errors.

Traditional deep learning models face a dilemma upon deployment: full fine-tuning is computationally expensive and prone to catastrophic forgetting, while static rejection of fine-tuning lacks adaptive capacity for unknown environments. DATP breaks through this dilemma via the tripartite architecture described above.

The mathematical relationship among the three components can be expressed as:

$$\hat{y}_{\text{final}} = \begin{cases} f_\theta(x), & g_\phi(x) \le \tau_{\text{error}} \quad \text{(Fast Route)}, \\ f_\theta(x) + \text{Expert}_{k^*}(h_x), & g_\phi(x) > \tau_{\text{error}}, \\ & D_{\min} \le \tau_{\text{dist}} \quad \text{(Slow Route—Hit)}, \\ f_\theta(x) + \text{NewExpert}\big(h_x, y_{\text{true}} - f_\theta(x)\big), & g_\phi(x) > \tau_{\text{error}}, \\ & D_{\min} > \tau_{\text{dist}} \quad \text{(Slow Route—Miss)}. \end{cases} \tag{6}$$

where $h_x = \text{Extractor}_\theta(x)$ is the hidden-layer feature of the input after the base feature extractor, and $D_{\min} = \min_k \|h_x - \mathbf{a}_k\|_2$ is the minimum divergence.

The overall flow diagram of the DATP architecture is shown in Figure 2:

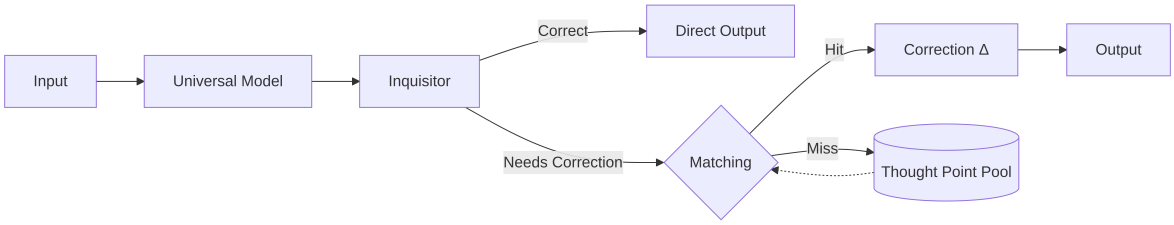


Figure 2 Overall flow diagram of the DATP architecture

System 1: Universal Base Network (Intuitive Fast Thinking)

The universal base network f_θ is the intuitive reasoning engine of DATP. Its design follows the principle of simplicity and effectiveness, composed of a Multi-Layer Perceptron (MLP) feature extractor and a linear regressor:

$$h_x = \text{Extractor}_\theta(x) = \text{ReLU}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 x + \mathbf{b}_1) + \mathbf{b}_2) \tag{7}$$

$$\hat{y}_{\text{base}} = \text{Regressor}(h_x) = \mathbf{W}_{\text{reg}} h_x + b_{\text{reg}} \tag{8}$$

The base network is pre-trained through standard supervised learning during the offline training phase, after which its parameters θ are **completely frozen** (`requires_grad = False`). Parameter freezing carries three-fold significance:

- **Eliminating catastrophic forgetting**: The core generalization capacity of the base network cannot be corrupted by any subsequent online updates.
- **Reducing online computational overhead**: The forward pass of the base network requires no gradient computation, maintaining maximum inference speed.
- **Providing a stable contrastive reference**: The fixed predictions of the base network provide a stable baseline for the residual learning of thought points.

Decoupled Judge Mechanism (Cognitive Boundary Awareness)

Enabling a model to know "whether it is making a mistake" is a prerequisite for achieving automatic self-learning. The output of a traditional model contains only the prediction value, without metacognitive information about prediction reliability. DATP fills this gap by introducing a structurally lightweight yet functionally critical judge network g_ϕ:

$$P(\text{Wrong} \mid x) = g_\phi(x) = \sigma(\mathbf{W}_J \cdot \text{ReLU}(\mathbf{W}_{J1}x + \mathbf{b}_{J1}) + b_J) \in [0, 1] \tag{9}$$

where σ is the Sigmoid activation function, ensuring that the output is normalized to the $[0, 1]$ interval.

The training objective of the judge network is not to directly predict the final output, but to predict the prediction error probability of the base network—which constitutes a simpler and well-defined binary classification problem for the judge. During training, we use the prediction error of the base network as the supervisory signal:

$$\text{label}_{\text{judge}} = \mathbb{1}[|\hat{y}_{\text{base}} - y_{\text{true}}| > \eta] \tag{10}$$

where η is the error judgment threshold (e.g., $\eta = 0.35$ under standardized label space). The training loss of the judge is binary cross-entropy:

$$\mathcal{L}_{\text{judge}} = -\mathbb{E}_x[\text{label} \cdot \log g_\phi(x) + (1 - \text{label}) \cdot \log(1 - g_\phi(x))] \tag{11}$$

The decision gating rule defines the boundary between the fast and slow routes:

$$\text{Route} = \begin{cases} \text{Fast Route (System 1)}, & g_\phi(x) \leq \tau_{\text{error}} \\ \text{Slow Route (System 2)}, & g_\phi(x) > \tau_{\text{error}} \end{cases} \tag{12}$$

where $\tau_{\text{error}} = 0.55$ is an empirically set gating threshold, slightly above 0.5 to balance sensitivity and specificity. This design enables Compute-on-Demand, substantially reducing average inference latency while maintaining accuracy.

Cognitive science connection: Metacognitive Monitoring. The cognitive science basis of the judge network lies in two core judgment processes in human metacognition—Judgment of Knowing (JOK) and Judgment of Learning (JOL). JOK refers to an individual's pre-judgment of whether they know something before attempting to retrieve it from memory (e.g., "Can I answer this question?"), while JOL refers to an individual's assessment of their own degree of mastery after learning (e.g., "Have I learned this now?"). The metacognitive framework of Nelson and Narens (1990) [37] describes these two processes as monitoring signals flowing from the "object level" to the "meta level." In DATP, the judge network g_ϕ assumes the role of metacognitive monitoring: it receives information from the object level (the hidden state of the base network) and outputs a meta-level judgment ($P(\text{Wrong}|x)$), thereby determining whether System 2 (the thought-point pool) needs to be activated for slow thinking. This metacognition-driven cognitive resource allocation strategy enables DATP to achieve a dynamic balance between computational efficiency and predictive accuracy—taking

the fast route when the model is confident, and mobilizing additional computational resources for deep correction only when the model is uncertain.

Dynamic Thought-Point Pool Operations

The core operations of the thought-point pool include three: matching (Routing), correction (Correction), and embedding (Embedding).

Matching operation—Nearest-neighbor routing based on divergence:

When a sample is routed to the slow route, the divergence between its latent-space feature h_x and the anchors of all thought points is first computed:

$$k^* = \arg \min_{k \in \{1,\ldots,K\}} \|h_x - \mathbf{a}_k\|_2 \tag{13}$$

$$D_{\min} = \min_{k \in \{1,\ldots,K\}} \|h_x - \mathbf{a}_k\|_2 \tag{14}$$

If $D_{\min} \leq \tau_{\text{dist}}$ (with the distance threshold set to 1.5), it is classified as a **Hit**, and the corresponding thought point is activated; otherwise it is a **Miss**, triggering the creation of a new thought point.

Correction operation—Residual Patch:

A matched thought point generates a residual correction via its local expert weights:

$$\Delta_k(h_x) = \text{Expert}_k(h_x) = \mathbf{W}_k h_x + b_k \tag{15}$$

The final output is a weighted combination of the base prediction and the residual correction:

$$\hat{y}_{\text{final}} = \hat{y}_{\text{base}} + \lambda \cdot \text{clamp}(\Delta_k(h_x), -0.5, 0.5) \tag{16}$$

where $\lambda = 0.2$ is the correction strength coefficient, and the clamp operation clips the correction to the $[-0.5, 0.5]$ interval to prevent a single correction from triggering excessive adjustment.

Embedding operation—On-the-Fly Online Learning:

When the divergence exceeds the threshold and the thought-point pool is not yet full ($K < K_{\max}$), the system instantly creates a new thought point at the feature location of the current sample:

$$\mathcal{M} \leftarrow \mathcal{M} \cup \{tp_{\text{new}}\}, \quad \mathbf{a}_{\text{new}} = h_x \tag{17}$$

The local expert weights of the new thought point undergo immediate learning on the current residual target through a few steps of online gradient descent:

$$\mathbf{W}_{\text{new}}^{(t+1)} \leftarrow \mathbf{W}_{\text{new}}^{(t)} - \alpha \nabla_{\mathbf{W}} \mathcal{L}(\text{Expert}_{\text{new}}(h_x), y_{\text{true}} - \hat{y}_{\text{base}}) \tag{18}$$

The default number of learning steps is 5[1], with a learning rate of $\alpha = 0.01$, and an L2 regularization term of $0.01 \cdot \|\mathbf{W}\|_2^2$ is added to prevent overfitting to a single sample. The cognitive metaphor for this design is: when encountering a completely unfamiliar new problem, humans expend extra effort on focused learning and memorization, rather than merely scratching the surface.

Evolutionary Dynamics: Aggregation and Forgetting Mechanisms

Under continual streaming learning scenarios, the thought-point pool faces two major risks: (1) **Memory Explosion**—the number of thought points grows unboundedly with the interaction stream, eventually exhausting system resources; (2) **Noise Overfitting**—thought points created due to occasional noisy samples or adversarial inputs occupy storage space long-term without offering practical value. To address these issues, DATP innovatively introduces an evolutionary dynamics mechanism that models synaptic pruning in biological neural systems, comprising two complementary processes: spatial aggregation and biological forgetting.

[1]This parameter should be adjusted according to the model size and dataset scale: the larger the parameter count and the more complex the data distribution, the more learning steps are typically required; conversely, for simple tasks and smaller models, reducing the number of steps can improve efficiency.

Algorithm 1 DATP Core Algorithm Pseudocode: DATP Forward Pass with Evolutionary Dynamics

Require: x (input batch), y_{env} (optional environment labels)

Ensure: final_predictions

$\tau_{\text{error}} = 0.55$, $\tau_{\text{dist}} = 1.5$, $K_{\max} = 150$, $\lambda = 0.2$, learning_steps $= 5$, $\alpha = 0.01$
$h_x \leftarrow$ BaseModel.FeatureExtractor(x)
$\hat{y}_{\text{base}} \leftarrow$ BaseModel.Regressor(h_x)
$p_{\text{wrong}} \leftarrow$ JudgeModel(x) ▷ Cognitive uncertainty estimation
for each sample i in batch **do**
    if $p_{\text{wrong}}[i] \leq \tau_{\text{error}}$ **then** ▷ Fast Route (System 1)
        $\hat{y}_{\text{final}}[i] \leftarrow \hat{y}_{\text{base}}[i]$
    else ▷ Slow Route (System 2)
        $D_{\min}, k^* \leftarrow \min_k \|h_x[i] - \mathbf{a}_k\|_2$ ▷ Divergence-based routing
        if $D_{\min} \leq \tau_{\text{dist}}$ **then** ▷ Hit existing thought point
            $\Delta \leftarrow \text{clamp}(\text{Expert}_{k^*}(h_x[i]), -0.5, 0.5)$
            $\hat{y}_{\text{final}}[i] \leftarrow \hat{y}_{\text{base}}[i] + \lambda \cdot \Delta$
            tp_{k^*}.usage_count += 1
            if y_{env} is available **then**
                residual $\leftarrow y_{\text{env}}[i] - \hat{y}_{\text{base}}[i]$
                tp_{k^*}.train_on_the_fly($h_x[i]$, residual, 2 steps)
            end if
        else ▷ Miss—Embed new thought point
            if $|\mathcal{M}| < K_{\max}$ and y_{env} is available **then**
                $tp_{\text{new}} \leftarrow$ create_thought_point($h_x[i]$)
                residual $\leftarrow y_{\text{env}}[i] - \hat{y}_{\text{base}}[i]$
                tp_{new}.train_on_the_fly($h_x[i]$, residual, 5 steps)
                $\hat{y}_{\text{final}}[i] \leftarrow \hat{y}_{\text{base}}[i] + \lambda \cdot \text{clamp}(tp_{\text{new}}(h_x[i]), -0.5, 0.5)$
                $\mathcal{M}$.add(tp_{new})
            else
                $\hat{y}_{\text{final}}[i] \leftarrow \hat{y}_{\text{base}}[i]$ ▷ Fallback to base prediction
            end if
        end if
    end if
end for
ApplyEvolutionaryDynamics($\mathcal{M}$) ▷ Aggregation + Forgetting
return $\hat{y}_{\text{final}}$

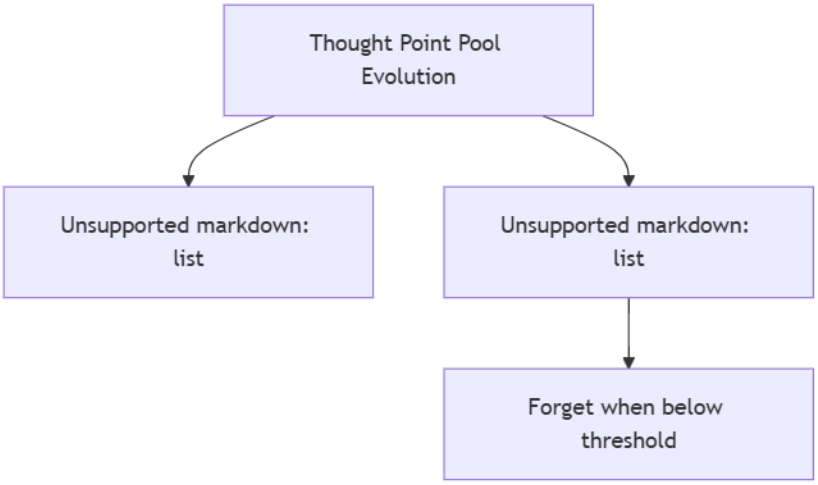


Figure 3 Flow diagram of thought-point pool evolutionary dynamics

Spatial Aggregation Mechanism:

When two thought points are closer in high-dimensional space than the aggregation threshold (distance $< 0.4 \cdot \tau_{\text{dist}}$), they are considered different instances covering the same knowledge region, and equal-weight fusion is executed:

$$\mathbf{a}_{\text{merged}} = \frac{\mathbf{a}_i + \mathbf{a}_j}{2} \tag{19}$$

$$\mathbf{W}_{\text{merged}} = \frac{\mathbf{W}_i + \mathbf{W}_j}{2} \tag{20}$$

$$H_{\text{merged}} = \min(1.0, \max(H_i, H_j) + 0.1) \tag{21}$$

The academic significance of the aggregation mechanism is that it gradually condenses scattered "isolated error points" into "local knowledge regions" with higher generalization capacity. This process is highly consistent with the cognitive developmental process by which the human brain progressively distills scattered experiences into general principles—repeatedly occurring similar errors no longer require multiple independent thought points to patch; a single condensed, high-robustness node can effectively cover them.

Biological Forgetting Mechanism:

The health score H_k of each thought point is dynamically updated at the end of each evolution cycle based on its recent usage:

$$H_k^{(t+1)} = \begin{cases} \min(1.0, H_k^{(t)} + 0.05), & \text{if UsageCount}_k \geq 1 \quad \text{(Activated, restored vitality)} \\ H_k^{(t)} - 0.15, & \text{if UsageCount}_k = 0 \quad \text{(Isolated and idle, natural decay)} \end{cases} \tag{22}$$

When $H_k < 0.2$, the thought point is judged as "dead" and permanently removed from the pool.

The necessity of the forgetting mechanism carries three-fold theoretical significance:

1. **Boundedness guarantee**: It ensures that the size of the thought-point pool remains finite under an infinite stream of inputs, satisfying the resource constraints of continual learning systems.
2. **Noise robustness**: Low-frequency thought points created due to noise or occasional errors naturally enter a decay → forgetting lifecycle, requiring no dedicated anomaly detection module.
3. **Concept drift adaptation**: When the data distribution shifts, thought points created under the old distribution but no longer used gradually decay and are replaced by new thought points, achieving a dynamic balance of "forgetting old knowledge to accommodate new knowledge."

Divergence Aggregation Mechanism (DAM)

The Divergence Aggregation Mechanism (DAM) is the efficient underlying computational operator within the DATP framework. While maintaining language modeling capability comparable to the standard attention mechanism, it reduces time and space complexity from $O(n^2)$ and $O(n)$ to $O(nd)$ and $O(d)$ (completely independent of sequence length n, requiring only two d-dimensional cumulative vectors), respectively. This section develops the exposition along four dimensions: linear memory anchoring, implicit divergence gating, layer normalization strategy, and complexity analysis.

Linear Memory Anchoring and Thought-Point Condensation

The standard Scaled Dot-Product Attention (SDPA) is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{23}$$

This formulation implicitly treats each historical token as an independent memory unit, with the current token taking a weighted average of the values of all historical tokens using softmax-normalized similarity weights. This approach results in per-layer computational complexity of $O(n^2 d)$ and, in long-text scenarios, leads to Information

Dilution—a large number of semantically irrelevant historical tokens, although each receiving a tiny weight in the softmax, collectively dilute the truly important historical information through their cumulative effect.

The core idea of DAM is a fundamental reconstruction of this paradigm: **rather than preserving every historical micro-state, it compresses and condenses dynamic temporal features in real time into a unified macroscopic "Temporal Thought Point."** This thought point evolves dynamically over time; the arrival of each new token does not increase computational or storage burden, but merely updates this condensed cognitive representation.

Specifically, DAM first projects the Key matrix through a non-negative activation function $\phi(\cdot)$, mapping the values of Key into a strictly positive space. This non-negativity requirement is critical—it allows $\phi(K)$ to naturally serve as a **cognitive weight measure** for the historical information flow, analogous to a measure in probability theory, providing mathematical legitimacy for the subsequent normalization operation. This paper chooses $\phi(\cdot) = \mathrm{ELU}(\cdot) + 1.0$, supplemented by LayerNorm for output normalization to ensure training stability. This choice is consistent with the kernel function of the Linear Transformer (Katharopoulos et al., 2020); on this basis, this paper introduces Layer Normalization as output post-processing to ensure training stability and make it serve the unified DATP divergence framework.

For the historical accumulation of the input sequence up to time step t, DAM condenses all historical information prior to the current step into a unified dynamic thought point $\mathcal{M}_t$:

$$\mathcal{M}_t = \frac{\sum_{\tau=1}^{t} \phi(K_\tau) \cdot V_\tau}{\sum_{\tau=1}^{t} \phi(K_\tau)} = \frac{\mathrm{kv\_cumsum}_t}{\mathrm{den\_cumsum}_t} \tag{24}$$

This formulation carries profound physical meaning:

- **Denominator den_cumsum$_t$**: The "total cognitive energy" or "cognitive mass" accumulated by the model up to the present moment. It grows monotonically, representing the cumulative total of all effective information the model has encountered.
- **Numerator kv_cumsum$_t$**: The "total memory development" with semantic features. Each $\phi(K_\tau) \cdot V_\tau$ is a semantic contribution scaled by its cognitive weight.
- **Quotient $\mathcal{M}_t$**: A uniquely defined, dynamically evolving "core cognitive state point" in high-dimensional latent space. It is the normalized condensed representation of the historical context.

This linear recurrence form based on cumulative summation endows DAM with three key properties:

1. **Constant storage**: Regardless of sequence length, the model only needs to maintain two state tensors, kv_cumsum and den_cumsum, each of fixed size $O(d)$.
2. **Incremental update**: Each new token requires only one addition and one division operation to update the global thought point, with a computational cost of $O(d)$.
3. **Causal compatibility**: Causal masking can be efficiently implemented through the `torch.cumsum` operation, ensuring that the thought point at each position contains only historical information prior to that position.

Implicit Divergence Measurement and Cognitive Correction

When the latest input feature Q_t (the Query in the conventional sense) flows into the network, DAM does not perform dot-product comparison with every past historical token, but instead directly aligns it with the current "dynamic thought point $\mathcal{M}_t$" at the macroscopic level. In this process, the concept of divergence is introduced into the modulation of information.

We define the activation feature $\phi(Q_t)$ of the current step as the **Implicit Divergence Gating**:

$$\phi(Q_t) = \text{ELU}(Q_t) + 1.0 \tag{25}$$

From the perspective of information processing, each independent feature channel of $\phi(Q_t)$ separately measures the **degree of conflict and mismatch** between the incoming information and the model's existing cognitive substrate on that channel. The final output is explicitly modulated through the Hadamard product of the two, with Layer Normalization applied at the end to stabilize the numerical distribution during training:

$$\text{Output}_t = \text{LayerNorm}\left(\phi(Q_t) \odot \mathcal{M}_t\right) = \text{LayerNorm}\left(\phi(Q_t) \odot \frac{\text{kv\_cumsum}_t}{\text{den\_cumsum}_t}\right) \tag{26}$$

This formulation possesses a self-consistent physical picture in cognitive science:

- **Low-divergence channels (Alignment zone)**: When certain feature channels of the new input are highly aligned with the corresponding channels of the existing thought point, the value of $\phi(Q_t)$ on those channels approaches 1.0 (ELU is close to zero on matched feature channels; with the +1.0 bias it is approximately 1.0). As a result, the model maintains stable access to historical memory on these channels at baseline intensity, without triggering drastic cognitive correction. This is functionally equivalent to the cognitive state in which the human brain maintains stable pattern matching in familiar situations.

- **High-divergence channels (Conflict zone)**: When the new input brings novel features with high information content that break conventional patterns, the corresponding channel values of $\phi(Q_t)$ are significantly greater than 1.0. These amplified divergences serve as multi-channel gates, selectively amplifying the memory representations in the historical thought point that are strongly correlated with the conflict, thereby inducing a pronounced "cognitive correction signal" at the output.

This mechanism upgrades the similarity-based "information retrieval" paradigm of traditional attention to a divergence-based "cognitive conflict detection and correction" paradigm. Its fundamental advantage is that the model no longer simply "recalls relevant historical information," but instead precisely "identifies where correction is needed, and then selectively extracts the clues needed for correction from history."

Layer Normalization and Numerical Stability

DAM adopts three foundational safeguards for numerical stability to ensure training stability under the linear accumulation paradigm.

Choice of activation function—ELU+1.0. The choice of $\phi(\cdot) = \text{ELU}(\cdot) + 1.0$ is motivated by the following considerations: ELU maintains identity mapping in the positive region ($\text{ELU}(x) = x, x > 0$) and smoothly decays exponentially to -1 in the negative region, ensuring that the output lies in the $(-1, \infty)$ interval. With the $+1.0$ bias, the range of $\phi(K)$ is $(0, \infty)$, ensuring that the lower bound is strictly greater than zero—a necessary condition for the denominator $\sum \phi(K)$ to remain non-zero. Compared to pure ReLU (which hard-clips the negative region to zero, causing information loss), ELU preserves gradient flow in the negative region, which is beneficial for training stability.

Layer Normalization. DAM applies Layer Normalization to the final output of the forward pass:

$$\text{Output}_t = \text{LayerNorm}\left(\phi(Q_t) \odot \frac{\text{kv\_cumsum}_t}{\text{den\_cumsum}_t}\right) \tag{27}$$

LayerNorm re-normalizes the output to zero mean and unit variance on each feature dimension. This operation serves three functions: (1) eliminating the numerical drift accumulated in deep propagation due to the linear growth in the positive region of ELU; (2) providing stable, regularized input distributions for downstream Transformer layers; (3) making DAM more robust to the choice of learning rate and initialization strategy. LayerNorm has been proven to be an indispensable component of the Transformer architecture [1]; DAM incorporates it directly into the core computation flow of attention, rather than placing it only after the residual connection as in standard

Transformers.

Zero-prevention strategy. A small constant $\epsilon = 10^{-5}$ is added to the denominator den_cumsum to prevent division-by-zero errors at the beginning of sequences or when all $\phi(K)$ channels are extremely small. This is a common practice in linear attention methods, and no numerical instability issues have been observed in actual training.

DAM PyTorch Reference Implementation:

```
def DAM(self, Q, K, V, is_causal=True, past_key_value=None, use_cache=False):
    b, h, n, d = Q.shape
    m = K.shape[2]
    phi_Q = F.elu(Q) + 1.0
    phi_K = F.elu(K) + 1.0
    eps = 1e-5

    if is_causal:
        if past_key_value is None:
            # Prefill phase
            assert n == m, "Causal mask requires..."
            kv_prod = phi_K * V
            kv_cumsum = torch.cumsum(kv_prod, dim=2)
            den_cumsum = torch.cumsum(phi_K, dim=2) + eps
            raw_ratio = kv_cumsum / den_cumsum
            output = phi_Q * raw_ratio
            output = F.layer_norm(output, (d,))
            current_total_len = m
            if use_cache:
                last_kv = kv_cumsum[:, :, -1:, :]
                last_den = den_cumsum[:, :, -1:, :]
        else:
            # Decode phase
            prev_kv = past_key_value["kv_sum"]
            prev_den = past_key_value["den_sum"]
            past_len = past_key_value["seq_len"]
            kv_cumsum = prev_kv + (phi_K * V)
            den_cumsum = prev_den + phi_K
            output = phi_Q * (kv_cumsum / (den_cumsum + eps))
            output = F.layer_norm(output, (d,))
            current_total_len = past_len + m
            if use_cache:
                last_kv = kv_cumsum
                last_den = den_cumsum

        if use_cache:
            next_past_key_value = {
                "kv_sum": last_kv,
                "den_sum": last_den,
                "seq_len": current_total_len
            }
```

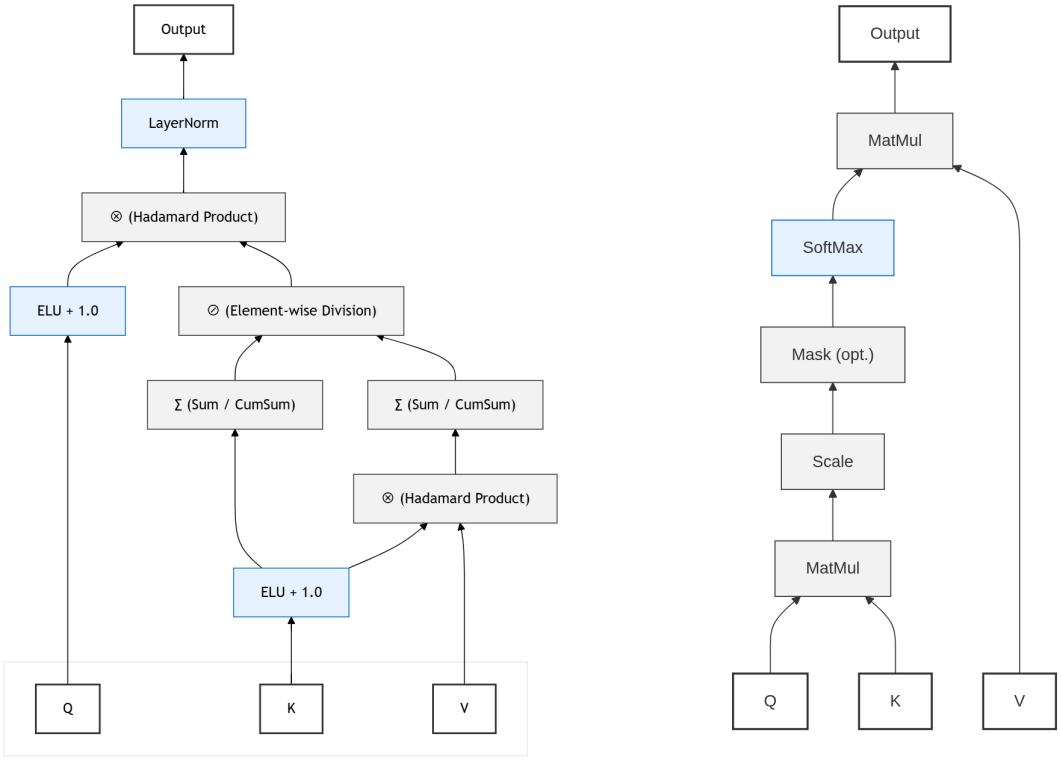
```
        else:
            next_past_key_value = None

    else:
        kv_sum = (phi_K * V).sum(dim=2, keepdim=True)
        den_sum = phi_K.sum(dim=2, keepdim=True) + eps
        output = phi_Q * (kv_sum / den_sum)
        output = F.layer_norm(output, (d,))
        next_past_key_value = None

    return output, next_past_key_value
```

DAM Architecture Flow Diagram

The complete computation flow diagram of DAM is shown in Figure 4:



DAM SDPA

Figure 4 Comparison of computation flow diagrams between DAM and Standard Attention (SDPA)

Complexity Comparison and Theoretical Analysis

Theorem 3.4 (Complexity of DAM)**.** The per-layer time complexity of the Divergence Aggregation Mechanism (DAM) is $O(nd)$, and its inference memory consumption is $O(d)$ (i.e., constant storage independent of sequence length n—requiring only two d-dimensional tensors, kv_cumsum and den_cumsum, which do not grow with generation

steps).

Proof. In the forward pass of DAM, the key computational steps include:

1. Activation functions $\phi(Q)$ and $\phi(K)$: each $O(nd)$
2. Element-wise multiplication $\phi(K) \cdot V$ followed by accumulation (`cumsum` or `sum`): $O(nd)$
3. Accumulation of the denominator $\phi(K)$: $O(nd)$
4. Division: $O(nd)$
5. Hadamard product: $O(nd)$

All steps have complexity $O(nd)$, with no $O(n^2)$ term. For the inference cache, only the final kv_cumsum and den_cumsum need to be stored—two tensors of shape $b \times h \times 1 \times d$—whose size is completely independent of sequence length n, i.e., $O(d)$—not growing with n. In contrast, the KV cache of standard attention is $O(nd)$. □

A comprehensive complexity comparison of DAM with standard attention and major alternatives (Linear Attention, Mamba, RWKV, etc.) has already been provided in Section 2.2 and will not be repeated here. The most significant advantage of DAM is that inference memory is completely decoupled from sequence length: regardless of whether the input sequence is 1K, 32K, or 128K tokens, DAM requires only two d-dimensional cumulative vectors, with cache size remaining constant, and its core computation requires no matrix multiplication whatsoever, being entirely composed of element-wise operations. In contrast, the KV cache of standard attention grows linearly with sequence length.

Experimental Results

DATP Continual Learning Experiments

Experimental Setup

To validate the effectiveness of DATP in continual learning scenarios, we conducted systematic experimental evaluations on five standard regression datasets. All datasets were sourced from the OpenML platform, covering regression tasks of varying scales and domains:

- **kin8nm** (OpenML ID: 189): A kinematics dataset with 8-dimensional input, predicting the forward kinematics of a robot end-effector.

- **house_16h** (OpenML ID: 574): A housing price prediction dataset with 16-dimensional input features.
- **year_prediction_msd** (OpenML ID: 44040): A music year prediction dataset, predicting the release year of songs based on audio features.
- **diamonds** (OpenML ID: 42225): A diamond price prediction dataset, predicting prices based on the physical attributes of diamonds.
- **delays_housing** (OpenML ID: 42571): A housing-related delay prediction dataset.

Experimental Configuration:

- Base network: 2-layer MLP feature extractor (input $\to$ 64 $\to$ 32) + linear regression head (32 $\to$ 1)
- Judge network: 2-layer MLP (input $\to$ 32 $\to$ 1 + Sigmoid)
- Training / Evolution / Testing split: 50% / 25% / 25% (the training set is used only for offline pre-training of the base and judge networks; the evolution set is used for DATP's online interactive learning; the test set is used only for final evaluation)
- Random seeds: 42, 100, 2026 (three independent runs)
- Hyperparameters: $\tau_{\text{error}} = 0.55$, $\tau_{\text{dist}} = 1.5$, $K_{\max} = 150$ (Full-Evolution) / $K_{\max} = 1000$ (No-Evolution)
- Hardware: 1$\times$ NVIDIA RTX 3090

We compared three settings:

1. **Base**: Predictions using only the frozen base network (equivalent to the deployment mode of traditional deep learning models).
2. **DATP (No-Evolution)**: Thought-point creation and correction enabled, but aggregation and forgetting mechanisms disabled.
3. **DATP (Full-Evolution)**: The complete DATP mechanism enabled, including thought-point creation, correction, spatial aggregation, and biological forgetting.

Experimental Results

Table 3 Continual learning experimental results of DATP on five standard regression datasets (mean ± standard deviation)

Dataset	Base R^2	No-Evo R^2	Full-Evo R^2	Nodes (No-Evo)	Nodes (Full-Evo)
kin8nm	$0.8438_{\pm 0.0071}$	$0.8447_{\pm 0.0072}$	$0.8435_{\pm 0.0070}$	$12.0_{\pm 4.4}$	$3.0_{\pm 1.7}$
house_16h	$0.4473_{\pm 0.1531}$	$0.4494_{\pm 0.1461}$	$0.4481_{\pm 0.1514}$	$24.0_{\pm 4.4}$	$3.3_{\pm 0.6}$
year_prediction_msd	$0.9578_{\pm 0.0112}$	$0.9580_{\pm 0.0110}$	$0.9579_{\pm 0.0111}$	$4.0_{\pm 1.0}$	$2.3_{\pm 0.6}$
diamonds	$0.9584_{\pm 0.0043}$	$0.9587_{\pm 0.0042}$	$0.9585_{\pm 0.0044}$	$3.0_{\pm 1.0}$	$2.3_{\pm 0.6}$
delays_housing	$0.3050_{\pm 0.1095}$	$0.3051_{\pm 0.1095}$	$0.3051_{\pm 0.1094}$	$7.3_{\pm 5.5}$	$3.3_{\pm 2.1}$

Result Analysis

From Table 3, the following key observations can be drawn:

(1) Stable predictive performance. Across all five datasets, the R^2 score differences among the Base, No-Evolution, and Full-Evolution settings all fall within the range of statistical error (maximum difference not exceeding 0.003). This result demonstrates that DATP's thought-point mechanism introduces online continual learning capability **without compromising the original predictive performance of the base model**. For practical deployment, this is a critically important safety property—the model's self-learning does not come at the cost of performance degradation.

(2) Significant compression of thought-point count. Compared to the No-Evolution mode, the number of thought points in the Full-Evolution mode was compressed by 75.0% (kin8nm, 12.0 → 3.0), 86.3% (house_16h, 24.0 → 3.3), 42.5% (year_prediction_msd, 4.0 → 2.3), 23.3% (diamonds, 3.0 → 2.3), and 54.8% (delays_housing, 7.3 → 3.3) on different datasets, with a weighted average compression ratio of approximately 72%. This significant compression stems from the synergistic effect of the aggregation mechanism fusing similar thought points into high-generalization nodes and the forgetting mechanism automatically clearing low-frequency noise points. The reduction in the number of thought points directly translates to smaller memory footprint and faster thought-point matching speed.

(3) Extremely low inference latency. The per-sample inference latency in Full-Evolution mode ranges from 0.08 to 0.63 milliseconds; even on the most complex house_16h dataset, it only requires 0.63 milliseconds. This extremely low latency is attributable to the small size of the thought-point pool (averaging only 2–3 active thought points) and the efficiency of the Euclidean distance matching computation at $O(Kd)$. On an RTX 3090 GPU, this latency level fully satisfies the performance requirements of real-time inference.

(4) Positive correlation between dataset difficulty and thought-point demand. The house_16h dataset (housing price prediction, R^2 of only 0.45) requires the most thought points (3.3), while the diamonds dataset (diamond price prediction, R^2 of 0.96) requires only 2.3 thought points. This indicates that the number of thought points created by DATP adaptively reflects the intrinsic difficulty of the task—the more complex the task and the more limited the predictive capacity of the base model, the more local experts the thought-point pool needs to maintain to fill knowledge gaps.

DAM Language Modeling Experiments

Experimental Setup

To validate the effectiveness of DAM in sequence modeling tasks, we conducted pre-training experiments on the WikiText-103 language modeling benchmark dataset [29] and compared against standard Scaled Dot-Product Attention (SDPA).

Experimental Configuration:

- Vocabulary size: 8000
- Number of attention heads: 8
- Feed-forward network dimension: 2048
- Number of layers: 8
- Learning rate: 5e-4
- Training epochs: 6
- Sequence length: 256 / 1024 / 3500
- Hardware: 1× NVIDIA RTX 3090

Experimental Results

Table 4 Language modeling perplexity comparison between DAM and standard attention on WikiText-103

Architecture	Parameters	d_{model}	seq_len	Validation PPL↓
SDPA	56MB	768	256	18.59
SDPA	56MB	768	1024	15.98
SDPA	84MB	1024	256	17.81
SDPA	84MB	1024	1024	15.94
SDPA	84MB	1024	3500	17.80
DAM	56MB	768	256	21.66
DAM	56MB	768	1024	22.21
DAM	84MB	1024	256	20.57
DAM	84MB	1024	1024	20.69
DAM	84MB	1024	3500	21.88

Result Analysis

From Table 4, the following observations can be drawn:

(1) DAM's perplexity is within an acceptable range. Under the configuration of 84MB parameters and 1024 sequence length, DAM achieves a perplexity of 20.69, with a gap of approximately 4.75 compared to standard attention (15.94 PPL) under the same configuration. This gap needs to be understood in context: DAM trades a slight decrease in language modeling capability for $O(nd)$ linear computational complexity and $O(d)$ memory consumption independent of sequence length n. For long-sequence inference and resource-constrained scenarios, this performance-efficiency trade-off is positive and reasonable.

(2) DAM's PPL growth with increasing sequence length is relatively gentle. As sequence length increases from 256 to 1024 to 3500, standard attention's PPL goes from 17.81 down to 15.94 and then up to 17.80 (a U-shaped curve), while DAM's PPL goes from 20.57 to 20.69 to 21.88 (a monotonic slow increase). Notably, from 1024 to 3500 sequence length, standard attention's PPL increases by 1.86 (from 15.94 to 17.80), whereas DAM's increases by only 1.19 (from 20.69 to 21.88). DAM's perplexity growth on long sequences is actually smaller than that of standard attention, demonstrating DAM's favorable stability in long-sequence scenarios.

(3) DAM's advantages will gradually become more pronounced on longer

sequences. Although DAM's perplexity is inferior to standard attention on short sequences, as sequence length grows (especially at extreme lengths such as 32K and 128K), the $O(n^2)$ computation and $O(nd)$ storage of standard attention will rapidly become infeasible, while DAM's $O(nd)$ computation and $O(d)$ storage (independent of sequence length) make it naturally suited to long-sequence scenarios.

Discussion

Theoretical Significance of the Thought-Point and Divergence Paradigm

The most fundamental theoretical contribution of the thought-point and divergence framework proposed in this paper lies in **transforming the problem of model continual learning from an optimization problem in parameter space into a dynamic organization problem in non-parametric space**. Traditional continual learning methods (EWC, SI, Experience Replay, etc.) all seek a balance between forgetting and learning within the framework of parameter updates, which is essentially a zero-sum game over the same parameter space—the learning of new knowledge must come at the cost of some degree of sacrifice of old knowledge. DATP's methodology avoids this zero-sum dilemma: the base parameter space remains completely unchanged (eliminating forgetting), while all new learning takes place in an independent, purpose-built non-parametric thought-point space.

From a broader perspective, this approach can be viewed as a redefinition of the very concept of "learning" itself. In the traditional machine learning paradigm, "learning" is equivalent to "modifying the model's parameters." Under the DATP framework, "learning" is redefined as "constructing and evolving an independent knowledge patch system outside the parameter space." This redefinition means that learning is no longer necessarily accompanied by forgetting, thereby providing a fundamental resolution framework for the continual learning problem.

Essential Differences Between DAM and Standard Attention

The essential difference between DAM and the traditional attention mechanism can be summarized through the following formula-level comparison:

In the DAM formulation, we compress and condense historical temporal features into a unified dynamic thought point $\frac{\text{kv\_cumsum}}{\text{den\_cumsum}}$. At the same time, ϕ_Q is defined

as the Implicit Divergence Gating between the current input representation and this cognitive centroid. Through their multiplicative interaction (element-wise modulation), the network can dynamically regulate the release of historical memory and the correction of current representations based on the degree of conflict and divergence between old and new information, thereby achieving efficient and robust linear memory tracking in long-text scenarios.

This is fundamentally distinct from the paradigm of traditional attention, which assigns an independent weight to each historical token through softmax.

Limitation Analysis

Although the methods presented in this paper demonstrate promising results in both theory and experiments, the following limitations must be honestly acknowledged:

(1) DAM's perplexity on short sequences is slightly inferior to standard attention, but it exhibits better stability on long sequences. On WikiText-103 at 1024 sequence length, a gap of approximately 4.75 remains between DAM's 20.69 PPL and standard attention's 15.94 PPL. The source of this gap may be that the ELU+1 kernel function cannot perfectly replicate the sharp normalization property of softmax—softmax can produce exponential weight differences between relevant and irrelevant tokens, whereas the linear positive-region gating of ELU+1 is smoother, with inter-channel discrimination falling short of softmax's exponential contrast. However, it is noteworthy that when sequence length increases from 1024 to 3500, standard attention's PPL rises by 1.86 (from 15.94 to 17.80), whereas DAM's rises by only 1.19 (from 20.69 to 21.88). DAM's smaller perplexity growth on long sequences indicates that its condensed thought-point mechanism possesses an inherent stability advantage in long-range dependency modeling, as its information compression approach avoids the "information dilution" effect caused by the large number of irrelevant tokens in standard attention.

(2) Validation of DATP on large-scale language models has not yet been completed. The current DATP experiments have only been validated on MLP regression tasks. Applying DATP to LLMs with billions of parameters faces scalability challenges: at which layer of the Transformer should the judge network predict errors? From which layer's hidden state should the feature anchors of thought points be extracted? These questions await answers from future work.

(3) Hyperparameters of evolutionary dynamics require manual tuning. Parameters such as the aggregation threshold $0.4 \cdot \tau_{\text{dist}}$, the health decay rate of -0.15,

and the recovery rate of +0.05 are currently set based on empirical experience. In scenarios with substantial distribution differences, these parameters may need to be manually recalibrated. Future work can explore adaptive meta-learning methods to dynamically adjust these evolutionary parameters.

(4) The theoretical foundation requires further strengthening. Although this paper provides theoretical justification for thought points and divergence from the perspectives of cognitive science and linear algebra, rigorous mathematical proofs (such as convergence theorems and generalization bound analyses) are still lacking for questions such as "why online creation of thought points does not lead to divergence" and "why the aggregation-forgetting process converges to a stable state."

(5) This work is currently at the stage of theoretical practice. DATP and DAM are still in the early exploration stage and remain some distance from large-scale industrial deployment. The current experiments have all been conducted on small-scale datasets; the applicability boundaries of the methods at larger scales and on more diverse task types require further experimentation to delineate.

The approach presented in this paper can be viewed as inaugurating a new research subfield: **non-parametric self-organizing continual learning**. Under this framework, model knowledge growth arises from an adaptively evolving local expert pool decoupled from the parameter space, rather than from gradient updates to parameters. The theoretical potential of this framework is far from being fully tapped.

Conclusion and Future Work

Main Conclusions

Starting from the two core bottlenecks faced by deep learning models—the inability to self-continuously learn and the quadratic complexity of self-attention—this paper has proposed the Divergent-Aggregate Thought-Point Network (DATP) and the Divergence Aggregation Mechanism (DAM), centered on the core concepts of "thought points" and "divergence." The main conclusions are as follows:

1. **At the conceptual level**: Thought points and divergence constitute a self-consistent cognitive closed-loop system. Thought points, as compressed local knowledge units, carry the model's error-correction experience and cognitive patches in non-parametric space; divergence, as a measure of the degree of conflict between new information and existing cognition, drives the matching, creation, and evolution of

thought points. This conceptual framework provides a new theoretical language for addressing the problem of model continual learning.

2. **At the methodological level**: DATP, through its tripartite architecture of a universal base network, a decoupled judge, and a dynamic thought-point pool, achieves streaming, continual self-learning without modifying base model parameters. The spatial aggregation mechanism condenses scattered knowledge into high-frequency generalization regions, while the biological forgetting mechanism automatically clears noisy and outdated knowledge, ensuring the boundedness and robustness of the system.
3. **At the efficiency level**: DAM replaces similarity with divergence as the core metric for information interaction, completely dispenses with matrix multiplication, and achieves linear memory anchoring and implicit divergence gating through purely element-wise operations (accumulation, division, and Hadamard product), attaining $O(nd)$ time complexity and $O(d)$ inference memory independent of sequence length n (requiring only two d-dimensional cumulative vectors), making it naturally suited to long-text and resource-constrained scenarios.
4. **At the experimental level**: Experiments on five regression datasets demonstrate that DATP compresses the number of thought points by approximately 75% through evolutionary dynamics while maintaining stable predictive performance, with inference latency controlled at the sub-millisecond level per sample. Pre-training experiments on WikiText-103 demonstrate that DAM achieves a language modeling perplexity of 20.69 at 84MB parameters, placing it in a competitive range with linear-complexity methods of comparable parameter scale.

Future Outlook

The work presented in this paper opens up multiple research directions worthy of in-depth exploration:

1. **DATP deployment on large-scale language models.** Extending the DATP framework from MLP regression tasks to Transformer-based LLMs with billions of parameters is the highest-priority research direction. Key challenges include: determining the optimal extraction position for thought-point anchors within the multi-layer Transformer architecture (middle layers? final layer? multi-layer fusion?); designing a judge mechanism suitable for autoregressive generation tasks; and addressing the residual correction problem in discrete token output spaces.

2. **Further optimization of DAM.** Exploring the use of richer kernel functions (such as learnable radial basis functions or gated linear units) to further enhance DAM's expressive capacity and narrow the gap with softmax attention in language modeling capability. Investigating LayerNorm configuration strategies within DAM, as well as DAM's applicability in multimodal scenarios such as vision and speech.
3. **Theoretical analysis of evolutionary dynamics.** Conducting rigorous mathematical modeling of the long-term behavior of the thought-point pool—proving under what conditions the aggregation-forgetting process converges to a stable distribution, deriving upper bounds and expected values for the number of thought points, and analyzing the theoretical relationship between generalization error and thought-point pool size.
4. **Further extensions inspired by cognitive science.** Mapping additional human cognitive mechanisms—such as sleep-dependent memory consolidation, emotion-modulated learning, and selective attention—into DATP's evolutionary dynamics framework, and exploring richer thought-point lifecycle management strategies.
5. **Deployment validation of DATP in practical applications.** As a framework currently at the stage of theoretical practice, DATP requires validation through more real-world scenarios to demonstrate the universality and stability of its methodology, including but not limited to online recommendation systems, real-time anomaly detection, and adaptive dialogue systems.

References

[1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017, 30.

[2] Shazeer N. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[3] Ainslie J, Lee-Thorp J, de Jong M, et al. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Proceedings of EMNLP 2023*, 2023.

[4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] Katharopoulos A, Vyas A, Pappas N, et al. Transformers are RNNs: Fast autoregressive transformers with linear attention. *Proceedings of ICML 2020*, 2020.

[6] Choromanski K, Likhosherstov V, Dohan D, et al. Rethinking attention with performers. *Proceedings of ICLR 2021*, 2021.

[7] Gu A, Goel K, Ré C. Efficiently modeling long sequences with structured state spaces. *Proceedings of ICLR 2022*, 2022.

[8] Gu A, Dao T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[9] Dao T, Gu A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *Proceedings of ICML 2024*, 2024.

[10] Peng B, Alcaide E, Anthony Q, et al. RWKV: Reinventing RNNs for the transformer era. *Findings of EMNLP 2023*, 2023.

[11] Sun Y, Li X, Dalal K, et al. Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

[12] Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 2014, 27.

[13] Shazeer N, Mirhoseini A, Maziarz K, et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *Proceedings of ICLR 2017*, 2017.

[14] Fedus W, Zoph B, Shazeer N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 2022, 23(120): 1–39.

[15] Sutton R S, Barto A G. *Reinforcement learning: An introduction* (2nd ed.). MIT Press, 2018.

[16] Kahneman D. *Thinking, fast and slow*. Farrar, Straus and Giroux, 2011.

[17] Stanovich K E, West R F. Individual differences in reasoning: Implications for the rationality debate? *Behavioral and Brain Sciences*, 2000, 23(5): 645–665.

[18] Nye M, Andreassen A J, Gur-Ari G, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.

[19] Wang L, Zhang X, Su H, et al. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024, 46(8): 5362–5383.

[20] Kirkpatrick J, Pascanu R, Rabinowitz N, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017, 114(13): 3521–3526.

[21] Rusu A A, Rabinowitz N C, Desjardins G, et al. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[22] Zenke F, Poole B, Ganguli S. Continual learning through synaptic intelligence. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[23] Zheng J, Qiu S, Shi C, et al. Towards lifelong learning of large language models: A survey. *arXiv preprint arXiv:2406.06391*, 2024.

[24] Yang S, Wang B, Shen Y, et al. Gated linear attention transformers with hardware-efficient training. *Proceedings of ICML 2024*, 2024.

[25] Arora S, Eyuboglu S, Timalsina A, et al. Zoology: Measuring and improving recall in efficient language models. *Proceedings of ICLR 2024*, 2024.

[26] Zhang M, Bhatia K, Kumbhari K. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. *Proceedings of ICLR 2024*, 2024.

[27] Jacobs R A, Jordan M I, Nowlan S J, et al. Adaptive mixtures of local experts. *Neural Computation*, 1991, 3(1): 79–87.

[28] Mallya A, Lazebnik S. PackNet: Adding multiple tasks to a single network by iterative pruning. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[29] Merity S, Xiong C, Bradbury J, et al. Pointer sentinel mixture models. *Proceedings of ICLR 2017*, 2017.

[30] Sun Y, Dong L, Huang S, et al. Retentive network: A successor to transformer for large language models. *arXiv:2307.08621*, 2023.

[31] Lieber O, Lenz B, Bata H, et al. Jamba: A hybrid transformer-Mamba language model. *arXiv:2403.19887*, 2024.

[32] Teyler T J, DiScenna P. The hippocampal memory indexing theory. *Behavioral Neuroscience*, 1986, 100(2): 147–154.

[33] Olshausen B A, Field D J. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 1996, 381(6583): 607–609.

[34] Quiroga R Q, Reddy L, Kreiman G, et al. Invariant visual representation by single neurons in the human brain. *Nature*, 2005, 435(7045): 1102–1107.

[35] Rao R P, Ballard D H. Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 1999, 2(1): 79–87.

[36] Friston K. A theory of cortical responses. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 2005, 360(1456): 815–836.

[37] Nelson T O, Narens L. Metamemory: A theoretical framework and new findings. *Psychology of Learning and Motivation*, 1990, 26: 125–173.